

Operating System (ISE3137)

Comprehensive Textbook & Lab Manual

Mastering OS Concepts, Ubuntu Administration & DevOps

Prof. Mehdi Pirahandeh

ISE Department
School of Global Convergence Studies
Inha University

"Jump Together, Fly Farther!"

This comprehensive manual bridges the gap between theoretical Operating System concepts and real-world system administration. It focuses purely on high-level conceptual understanding and practical Ubuntu/DevOps workflows, entirely bypassing low-level C programming and kernel internals to ensure a clear, process-driven learning experience.

March 29, 2026

Contents

1	Module 1: Fundamentals of Operating Systems	2
1.1	What is an Operating System?	2
1.2	System Services and System Calls (Conceptual)	2
2	Module 2: Process Management	3
2.1	The Concept of a Process	3
2.2	The Process Control Block (PCB)	3
2.3	Context Switching	3
2.4	Interprocess Communication (IPC)	4
3	Module 3: Threads and Concurrency	5
3.1	What is a Thread?	5
3.2	Concurrency vs. Parallelism	5
3.3	Amdahl's Law (Step-by-Step)	5
4	Module 4: Ubuntu Administration & DevOps Basics	7
4.1	The Linux Philosophy	7
4.2	The Linux File System Hierarchy	7
4.3	User and Permission Management (Lab Exercise)	7
4.4	Package Management (APT)	8
5	Module 5: Advanced Process Control & Job Scheduling	10
5.1	Signals: How the OS Communicates with Processes	10
5.2	Foreground, Background, and Job Control	11
5.3	Task Scheduling with Cron	11
6	Module 6: Systemd and Service Management	12
6.1	What is a Daemon?	12
6.2	The Role of Systemd (PID 1)	12
6.3	Managing Services with systemctl	12
7	Module 7: System Monitoring & Performance Tuning	13
7.1	The /proc Filesystem Illusion	13
7.2	Analyzing System Load	13
7.3	Real-Time Dashboards: top and htop	13
7.4	Memory Management and Swap	14

1 Module 1: Fundamentals of Operating Systems

1.1 What is an Operating System?

An Operating System (OS) is the foundational software that acts as an intermediary between the computer hardware and the computer user. Its primary goal is to make the computer system convenient to use while ensuring that the hardware resources are utilized efficiently.

Without an operating system, every user application would need to include its own instructions for talking to the hardware—meaning every programmer would have to know exactly how to spin a hard drive, send pixels to a monitor, and read voltages from a keyboard. The OS abstracts this immense complexity away.

Step-by-Step: The "Opening a Web Browser" Example

To understand the role of the OS, let's look at what happens conceptually when you double-click a web browser icon:

1. **User Input:** You move the mouse and double-click the icon. The mouse hardware sends an electronic signal to the OS.
2. **OS Translation:** The OS translates this electronic signal into coordinates on your screen, determining that you clicked the browser icon.
3. **Storage Access:** The OS locates the browser application files on your physical Hard Drive or SSD.
4. **Memory Allocation:** The OS allocates a block of RAM (Random Access Memory) and copies the browser's code from the storage drive into that RAM.
5. **CPU Execution:** The OS schedules the CPU to begin reading the browser's code from RAM, officially turning the "program" into a running "process."
6. **Display:** The OS provides the browser with a graphical window to draw its interface, sending the final rendering to your monitor.

All of this happens in milliseconds, completely invisibly to the user.

1.2 System Services and System Calls (Conceptual)

To interact with the OS, applications use **System Calls**. Think of system calls as an API (Application Programming Interface) for the hardware.

If a program wants to read a file, it does not communicate with the hard drive. Instead, it makes a *System Call* to the OS requesting the file. The OS checks if the program has permission, fetches the file, and hands the data back to the program. This protects the system from malicious or poorly written programs that might try to overwrite critical data.

2 Module 2: Process Management

2.1 The Concept of a Process

A **program** is a passive entity—a file sitting on your hard drive. A **process** is an active entity—a program that has been loaded into memory and is currently executing.

When you open three different tabs of the same web browser, you are running three separate *processes* from the same *program*.

The Five States of a Process

As a process executes, it moves through various states. Understanding these states is critical for DevOps engineers when troubleshooting frozen applications.

- **New:** The process is being initialized and loaded into memory.
- **Ready:** The process has everything it needs to run and is waiting in a queue for the CPU to become available.
- **Running:** The CPU is actively executing the instructions of this process.
- **Waiting (or Blocked):** The process cannot continue until an external event occurs (e.g., waiting for the user to type a password, or waiting for data to download from the network). It temporarily gives up the CPU.
- **Terminated:** The process has finished its execution or has been forcibly killed by the administrator.

2.2 The Process Control Block (PCB)

To manage hundreds of processes at once, the OS needs a way to keep track of them. It uses a data structure called the **Process Control Block (PCB)**. Think of the PCB as a passport or ID card for the process. It contains:

- The Process ID (PID).
- The current state (Ready, Running, Waiting).
- What memory it is allowed to access.
- What files it currently has open.

2.3 Context Switching

A standard CPU core can only execute one process at a time. To create the illusion that multiple programs are running simultaneously (Concurrency), the OS rapidly switches the CPU between different processes thousands of times per second.

This act of stopping one process, saving its current state into its PCB, loading the state of the next process from its PCB, and resuming execution is called a **Context Switch**. Context switching takes time and is considered pure overhead—the computer is doing administrative work rather than executing user tasks.

2.4 Interprocess Communication (IPC)

In modern systems, processes often need to talk to each other. For example, a web server process needs to talk to a database process. Because processes are isolated for security, they must use IPC.

Step-by-Step: IPC Models: Shared Memory vs. Message Passing

Imagine two workers in an office who need to collaborate on a document.

- **Shared Memory:** The two workers sit at the same desk and write on the exact same piece of paper.
 - *Pros:* Extremely fast.
 - *Cons:* They might bump hands or overwrite each other's words if they don't carefully synchronize (taking turns).
- **Message Passing:** The workers sit in different rooms and send emails to each other with updates.
 - *Pros:* Very safe. No risk of bumping hands. Works well even if the workers are in entirely different buildings (networked computers).
 - *Cons:* Slower due to the time it takes to write, send, and read the email.

3 Module 3: Threads and Concurrency

3.1 What is a Thread?

If a process is a factory, a **thread** is a worker inside that factory.

Creating a new process (building a new factory) is highly resource-intensive. It requires a new memory space, new security permissions, and a new PCB. However, if an application needs to do multiple things at once, it can create multiple **threads** inside its existing process. Threads share the same memory, code, and files, making them incredibly fast to create and switch between.

Real-World Example: A Word Processor

Imagine you are typing an essay in Microsoft Word. If Word was a single-threaded application, the moment you clicked "Save," the entire application would freeze until the file finished writing to the hard drive.

Instead, Word is multithreaded:

- **Thread 1:** Listens to your keyboard and displays the letters on the screen.
- **Thread 2:** Runs in the background, checking your spelling and underlining errors in red.
- **Thread 3:** Periodically saves a backup of your document to the disk.

Because they are threads, Thread 2 and Thread 3 can easily read the text that Thread 1 is typing without needing complex IPC.

3.2 Concurrency vs. Parallelism

These terms are often confused but mean different things in OS design:

- **Concurrency:** The illusion of simultaneous execution. On a single-core CPU, the OS rapidly context-switches between threads. They are not actually running at the exact same millisecond, but they are all making progress.
- **Parallelism:** True simultaneous execution. On a multi-core CPU, Thread A runs on Core 1 at the exact same physical moment that Thread B runs on Core 2.

3.3 Amdahl's Law (Step-by-Step)

When DevOps engineers provision servers, they often ask: *"If I double the number of CPU cores, will my application run twice as fast?"* Amdahl's law proves that the answer is usually **No**.

The speed of an application is limited by its *serial* portion—the part of the code that must run sequentially and cannot be split into threads.

Step-by-Step: Calculating Performance Gain

Suppose you have an application that takes 100 seconds to run.

- 25 seconds of the task is setup/cleanup that cannot be parallelized (**Serial Portion = 25% or 0.25**).
- 75 seconds of the task is data crunching that can be split infinitely (**Parallel Portion = 75% or 0.75**).

If you move from 1 core to 4 cores, how much faster will it run?

1. The 25 seconds of serial work still takes 25 seconds.
2. The 75 seconds of parallel work is divided by 4 cores = 18.75 seconds.
3. Total new time = $25 + 18.75 = 43.75$ seconds.

Your speedup is $100/43.75 = \mathbf{2.28x}$. Even though you quadrupled (4x) the hardware, you only got a 2.28x speedup because the 25% serial portion acts as a severe bottleneck.

4 Module 4: Ubuntu Administration & DevOps Basics

In the professional world, understanding Operating Systems theory must be paired with practical administration skills. Linux (specifically Ubuntu) is the industry standard for DevOps, cloud computing, and server hosting.

4.1 The Linux Philosophy

Linux is built on a few core principles that dictate how we interact with it:

1. **Everything is a file:** Hardware devices, network sockets, and running processes are all represented as text files in the file system.
2. **Small, single-purpose tools:** Instead of massive applications, Linux provides small command-line tools that do one thing perfectly (like **grep** for searching, or **cat** for reading).
3. **Chaining tools together:** We use "Pipes" (**|**) to send the output of one tool directly into the input of another.

4.2 The Linux File System Hierarchy

Unlike Windows, which uses drive letters like **C:**, Linux uses a unified directory tree starting at the "Root" directory, represented by a forward slash **/**.

- **/bin & /usr/bin:** Contains essential user command binaries (like **ls**, **cp**, **mkdir**).
- **/etc:** The central location for all system-wide configuration files. If you need to configure a web server or a database, you edit files here.
- **/var:** "Variable" data. This is where the OS stores data that grows and changes frequently, such as system logs (**/var/log**) and databases.
- **/home:** Where individual users' personal files and documents are stored.
- **/dev:** "Device" files. This represents your hardware. For example, your first hard drive might be represented as a file named **/dev/sda**.

4.3 User and Permission Management (Lab Exercise)

Linux is a multi-user operating system. Security is managed by assigning strict permissions to files and directories. Every file has three sets of permissions: **Read (r)**, **Write (w)**, and **Execute (x)**. These permissions are assigned to the **Owner**, the **Group**, and **Others** (everyone else).

DevOps/Admin Challenge: DevOps Task 1: Creating Users and Managing Permissions

Scenario: You are the systems administrator. You need to create a new developer account, give them a project directory, and ensure no one else can modify their code.

Execution Sequence:

```
1 # 1. Create a new user named 'dev_alice'
2 sudo adduser dev_alice
3
4 # 2. Create a project directory in the root drive
5 sudo mkdir /project_alpha
6
7 # 3. Change the ownership of the directory to Alice
8 sudo chown dev_alice:dev_alice /project_alpha
9
10 # 4. Modify permissions so ONLY Alice can read, write, and
    execute
11 sudo chmod 700 /project_alpha
12
13 # Verify the permissions
14 ls -ld /project_alpha
```

Explanation: `sudo` elevates your privileges to root. `chmod 700` sets Owner permissions to 7 (Read=4 + Write=2 + Execute=1), and Group/Others to 0 (No access).

4.4 Package Management (APT)

In Windows, you typically download an `.exe` file from a website to install software. In Ubuntu, software is managed securely through central repositories using the Advanced Package Tool (APT). APT automatically resolves dependencies (downloading other software required for your program to run).

Step-by-Step: Updating and Installing Software

A standard DevOps routine upon logging into a new Ubuntu server:

```
1 # 1. Update the local list of available software from the
    internet
2 sudo apt update
3
4 # 2. Upgrade all currently installed software to the latest
    versions
5 sudo apt upgrade -y
6
7 # 3. Install a new package (e.g., the Nginx web server)
8 sudo apt install nginx -y
9
10 # 4. Remove software that is no longer needed
```

```
11 | sudo apt remove apache2
```

5 Module 5: Advanced Process Control & Job Scheduling

As a system administrator or DevOps engineer, you will rarely launch programs by clicking icons. You will manage hundreds of concurrent processes from a text-based terminal. Mastering how to control, suspend, and schedule these processes is a mandatory skill.

5.1 Signals: How the OS Communicates with Processes

When you want a running process to stop, you don't just pull the power cord. The Operating System uses **Signals**—software interrupts sent to a program to indicate that an important event has occurred.

Think of a signal as a tap on the shoulder. Depending on the type of tap, the process will react differently.

- **SIGINT (Signal 2 - Interrupt):** Sent when you press **Ctrl+C** in the terminal. It politely asks the foreground process to terminate.
- **SIGTSTP (Signal 20 - Terminal Stop):** Sent when you press **Ctrl+Z**. It tells the process to pause (suspend) its execution immediately, but keeps it in memory.
- **SIGTERM (Signal 15 - Terminate):** The default signal sent by the **kill** command. It is a polite request to shut down. The process is allowed to save its data, close its open files, and exit gracefully.
- **SIGKILL (Signal 9 - Kill):** The "assassin" signal. This signal is *not* sent to the process; it is intercepted by the OS kernel, which immediately destroys the process. The process has no chance to save data. This is used only as a last resort for frozen applications.

DevOps/Admin Challenge: DevOps Task 2: Handling a Run-away Process

Scenario: A data-crunching script named **heavy_calc.sh** has frozen and is consuming 100% of the CPU. You need to stop it safely, and if it refuses, force it to stop.

Execution Sequence:

```
1 # 1. Find the Process ID (PID) of the script
2 ps aux | grep heavy_calc.sh
3
4 # Assume the output shows the PID is 4052.
5
6 # 2. Try the polite way first (SIGTERM)
7 kill 4052
8
9 # 3. Wait a few seconds. If it is still running, force kill it (
    SIGKILL)
```

```
10 | kill -9 4052
```

5.2 Foreground, Background, and Job Control

A standard terminal can only run one program in the **foreground** at a time. If you start a backup script that takes 3 hours to run, your terminal is locked for 3 hours. Job control solves this.

Step-by-Step: Mastering Terminal Job Control

You can push jobs to the **background**, allowing them to run invisibly while you continue typing new commands in the foreground.

1. **Start in the background:** Append an ampersand (&) to your command.
`tar -czf backup.tar.gz /var/www &`
2. **Pause a foreground job:** If you forgot the &, press **Ctrl+Z**. The terminal will say `[1]+ Stopped`.
3. **Resume in the background:** Type `bg %1`. The paused job will now continue running silently in the background.
4. **Bring it back:** Type `fg %1` to bring job number 1 back to the foreground so you can interact with it.

5.3 Task Scheduling with Cron

Servers run 24/7. Administrators do not stay awake to run database backups at 3:00 AM. They use a time-based job scheduler called **Cron**.

A **crontab** (cron table) is a text file where you define what scripts should run and exactly when they should trigger. The syntax uses five asterisks representing time units:

```
[Minute] [Hour] [Day of Month] [Month] [Day of Week] /path/to/command
```

Crontab Syntax Examples

- `0 3 * * * /backup.sh` → Runs at 3:00 AM every single day.
- `30 14 * * 5 /report.sh` → Runs at 2:30 PM (14:30) every Friday (Day 5).
- `*/15 * * * * /healthcheck.sh` → Runs every 15 minutes, infinitely.

To edit your user's crontab, simply type `crontab -e` in the terminal.

6 Module 6: Systemd and Service Management

6.1 What is a Daemon?

In OS terminology, a **daemon** is a background process that runs continuously, waiting to handle requests. Daemons do not have a graphical user interface. By convention, daemon names usually end with the letter 'd' (e.g., `sshd` for the Secure Shell Daemon, `httpd` for the Web Server Daemon).

6.2 The Role of Systemd (PID 1)

When a Linux system boots, the kernel initializes the hardware and then launches the very first process: **systemd** (Process ID 1). Systemd is the "mother of all processes." Its job is to read configuration files and sequentially start all the other daemons required to make the server functional (networking, logging, web servers, databases).

6.3 Managing Services with systemctl

DevOps engineers use the `systemctl` command to talk to systemd and manage these daemons.

Step-by-Step: Understanding Service States

There is a critical difference between a service's *current state* and its *boot state*.

- **Current State (Start/Stop):** Is the program running right now in RAM?
- **Boot State (Enable/Disable):** Should systemd automatically launch this program the next time the server reboots?

Essential Commands:

```
1 # Start the Nginx web server immediately
2 sudo systemctl start nginx
3
4 # Check if it is running healthily
5 sudo systemctl status nginx
6
7 # Tell the OS to start Nginx automatically on every future
  reboot
8 sudo systemctl enable nginx
9
10 # Reload the configuration files without dropping active user
    connections
11 sudo systemctl reload nginx
```

7 Module 7: System Monitoring & Performance Tuning

When a server becomes sluggish or crashes, a professional does not randomly restart services. They investigate. The OS provides precise mathematical metrics to identify the bottleneck: CPU, Memory, or Disk I/O.

7.1 The /proc Filesystem Illusion

To truly understand how an OS manages processes, we must look under the hood. In Linux, the `/proc` directory is not a real directory stored on your hard drive. It is a **pseudo-filesystem** maintained dynamically in RAM by the OS kernel.

It provides a window directly into the kernel's internal data structures. If you navigate to `/proc`, you will see numbered folders for every single running Process ID.

- `cat /proc/cpuinfo`: Displays hardware information about the CPU.
- `cat /proc/meminfo`: Shows real-time memory usage and swap space.

7.2 Analyzing System Load

The `uptime` command shows how long the system has been running, but more importantly, it shows the **Load Average** over the last 1, 5, and 15 minutes.

Understanding Load Average

Think of the CPU as a bridge, and processes as cars trying to cross it.

- A load of **0.00**: The bridge is completely empty.
- A load of **1.00**: The bridge is exactly full. Cars are crossing at maximum efficiency, but no one is waiting in line.
- A load of **5.00**: The bridge is full, and 4 cars are stuck in a traffic jam waiting to cross.

Crucial Note: This scale is per-core. If you have a 4-core CPU, a load of 4.00 means the system is at 100% capacity. A load of 12.00 on a 4-core system means severe processor starvation. By looking at the 1, 5, and 15-minute averages, you can tell if the traffic jam is getting worse or clearing up.

7.3 Real-Time Dashboards: `top` and `htop`

While `ps aux` gives you a static snapshot of processes, `top` provides a dynamic, constantly updating dashboard of system resource usage.

`htop` is a modern, colorful alternative to `top`. It allows you to scroll vertically to see all processes, scroll horizontally to see full command paths, and press `F9` to visually send kill signals to misbehaving programs without needing to memorize their PIDs.

7.4 Memory Management and Swap

When an OS runs out of physical RAM, it does not immediately crash. It uses **Swap Space**—a designated area on the hard drive acting as slow, emergency RAM.

You can monitor this using the `free -h` command.

```
1 $ free -h
2
```

	total	used	free	shared	buff/cache
	available				
3 Mem:	15Gi	8.2Gi	2.1Gi	110Mi	5.3Gi
	7.0Gi				
4 Swap:	4.0Gi	0.0Ki	4.0Gi		

If you see high Swap usage, it means your server is starved for memory and is constantly moving data between RAM and the Hard Drive (a process called *Thrashing*), which destroys performance.

Final Advice from Prof. Mehdi Pirahandeh:

"An Operating System is not just a piece of software; it is a meticulously managed ecosystem. To truly master it, you must move beyond simply executing programs. You must learn to observe their behavior, manipulate their scheduling, and trace their resource consumption through the terminal. True power in computing comes from understanding the bridge between the high-level application and the low-level hardware.

Jump Together, Fly Farther!"

To consolidate your learning, review these master tables connecting theoretical OS concepts with their practical DevOps terminal counterparts.

Theoretical OS Concept		Practical Implementation / Description
Process Creation (fork & exec)	DevOps Equivalent	The OS duplicating memory space to launch a new program.
		A bash shell spawning a new command in the terminal.
Process State Transition (Forced)	DevOps Equivalent	Moving from "Running" to "Terminated" without cleanup.
		Using the <code>kill -9 <PID></code> (SIGKILL) command.
Process State Transition (Graceful)	DevOps Equivalent	Allowing a process to save data/cleanup before exiting.
		Sending a SIGTERM via the standard <code>kill <PID></code> command.
Process Suspension	DevOps Equivalent	Moving from "Running" to "Waiting/Suspended" state.
		Pressing <code>Ctrl+Z</code> to send a SIGTSTP signal.
Zombie Process Resolution	DevOps Equivalent	A dead process waiting for its parent to read its exit status.
		Identifying 'Z' state in <code>top</code> and killing the parent process.
Concurrency via Time-Slicing	DevOps Equivalent	The OS rapidly switching tasks on a single core.
		Using <code>&</code> to run background jobs while typing in the foreground.
Multicore Scaling (Amdahl's Law)	DevOps Equivalent	Distributing parallel tasks across multiple CPU cores.
		Analyzing per-core Load Average using <code>htop</code> or <code>uptime</code> .
Local IPC (Pipes & Shared Mem)	DevOps Equivalent	Processes sharing data via unidirectional or shared conduits.
		Chaining terminal commands using the Pipe operator (<code> </code>).
Network IPC (Sockets & RPC)	DevOps Equivalent	Processes communicating across networks via endpoints.
		Checking open ports and connections using <code>lsof -i</code> or <code>netstat</code> .

Table 1: Ultimate Master Mapping of OS Theory to System Administration (Part 1)

Theoretical OS Concept	Practical Implementation / Description
The Process Control Block (PCB)	Kernel memory storing process state, registers, and metadata.
DevOps Equivalent	Viewing the <code>/proc/<PID>/status</code> virtual file directly.
System Calls & Kernel Interface	The bridge between user-space apps and protected hardware.
DevOps Equivalent	Tracing real-time application resource requests using <code>strace</code> .
Bootstrapping & Initialization	The OS launching critical services immediately after kernel load.
DevOps Equivalent	The <code>systemd</code> daemon (PID 1) and <code>systemctl enable</code> .
Daemons & Background Services	Infinite-loop background processes waiting for requests.
DevOps Equivalent	Managing web/database servers via <code>systemctl start/stop</code> .
Access Control & Security	OS restricting memory and file access to authorized users.
DevOps Equivalent	Managing user/group permissions via <code>chmod</code> and <code>chown</code> .
CPU Scheduling & Priority	The OS deciding which "Ready" process gets CPU time next.
DevOps Equivalent	Adjusting process priority/niceness using <code>nice</code> and <code>renice</code> .
Virtual Memory & Thrashing	The OS utilizing disk space when physical RAM is exhausted.
DevOps Equivalent	Monitoring Swap space usage and starvation via <code>free -h</code> .
Long-Term Scheduling	Deferring process execution to a highly specific future time.
DevOps Equivalent	Automating administrative scripts using <code>cron</code> and <code>crontab</code> .
Hardware Abstraction	The OS presenting physical hardware interfaces as standard files.
DevOps Equivalent	Interacting with physical storage drives via the <code>/dev</code> directory.
File System Hierarchy	Organizing data in an inverted tree with specialized branches.
DevOps Equivalent	Separating static configs (<code>/etc</code>) from dynamic logs (<code>/var</code>).

Table 2: Ultimate Master Mapping of OS Theory to System Administration (Part 2)